

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY COURSE
CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

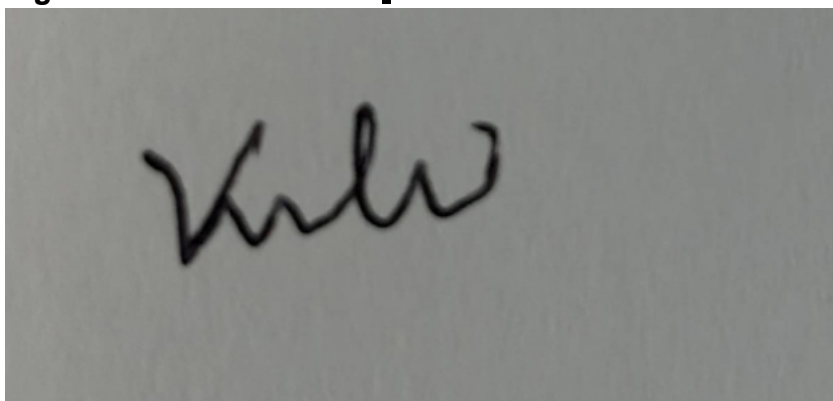
Total Marks: 50

Annexure A

Code of Conduct:

I _____VARSHA.S/192511358_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in black ink. The signature is stylized and appears to be 'V. S.' or similar, written on a light-colored background.

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.

Fix issues related to bitwise operations and endianness handling.

Optimize the implementation to improve processing speed for large files (≥ 50 MB).

Evaluate how secure hash functions improve integrity compared to basic checksum methods.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.

Optimize the handshake process to reduce latency.

Refactor the code to reuse session keys efficiently (session resumption).

Analyze the performance vs security trade-offs in TLS optimization.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.

Fix errors in signature generation and verification steps.

Optimize modular arithmetic operations for faster execution.

Evaluate the impact of randomness quality on signature security.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).

Fix issues related to key lifecycle management (generation, distribution, rotation).

Optimize secure storage using encryption and access control mechanisms.

Analyze how poor key management can compromise an entire cryptographic system.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.

Fix errors in packet handling and integrity verification.

Optimize throughput for large file transfers (≥ 100 MB).

Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with	Code is mostly clear with minor documentatio	Code lacks structure and proper documentatio	Poorly written code with no documentati

		proper comments and	n gaps	n	on
		documentatio n			

- **1. Debugging Hash Function Implementation (SHA Family)**
Introduction

A Secure Hash Algorithm (SHA) generates a fixed-length hash value from an input message. The same input should always produce the same hash. If inconsistent outputs occur, there are implementation errors in padding, block processing, or bitwise operations.

1. Identify Errors

- Incorrect message padding.
- Wrong calculation of message length.
- Improper splitting into 512-bit blocks.
- Incorrect bitwise rotation and shift operations.
- Endianness mismatch while reading bytes.

2. Fix the Issues

- Apply SHA padding correctly by adding a '1' bit followed by zeros.
- Append the original message length as a 64-bit integer.
- Process every block in the correct order.
- Use proper left rotations instead of simple shifts.
- Maintain big-endian byte order throughout processing.

3. Optimization

- Read large files in chunks instead of loading the whole file.
- Reuse memory buffers.
- Minimize unnecessary conversions.
- Optimize loop execution for message schedule generation.
- Use efficient bitwise operations.

4. Security Comparison

Secure Hash (SHA)	Basic Checksum
--------------------------	-----------------------

Cryptographically secure	Not secure
--------------------------	------------

Detects tampering	Detects only accidental errors
-------------------	--------------------------------

Collision resistant	Easy to forge
---------------------	---------------

Used in digital signatures	Used for error checking
----------------------------	-------------------------

Conclusion

Correct implementation of padding, block processing, and bitwise operations ensures consistent hash values. Optimized SHA implementations provide fast processing while maintaining data integrity and strong security.

• 2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

Introduction

SSL/TLS provides secure communication by encrypting data exchanged between clients and servers. Handshake failures and delays reduce security and performance.

1. Debug Certificate Validation

- Verify certificate expiration date.
- Check trusted Certificate Authority (CA).
- Validate certificate chain.
- Verify server hostname.
- Reject self-signed or invalid certificates.

2. Debug Key Exchange

- Ensure both client and server use compatible cipher suites.
- Verify public/private key operations.
- Generate session keys correctly.

- Prevent handshake timeout errors.

3. Optimization

- Enable TLS Session Resumption.
- Cache previously verified certificates.
- Reuse session keys securely.
- Reduce unnecessary cryptographic operations.
- Use efficient cipher suites.

4. Performance vs Security

Performance Improvement Security Impact

Faster handshake	Slight increase in session reuse risk
Session resumption	Reduced computation
Certificate caching	Faster authentication
Strong encryption	Higher CPU usage

Conclusion

Proper certificate validation and optimized handshake mechanisms improve both security and communication speed without compromising confidentiality.

• 3. Debugging and Optimizing Digital Signature Algorithm (DSA)

Introduction

The Digital Signature Algorithm (DSA) ensures message authenticity, integrity, and non-repudiation. Invalid signatures occur due to incorrect parameters or weak random number generation.

1. Identify Issues

- Incorrect prime numbers (p , q).
- Invalid generator value (g).
- Weak random number (k).
- Incorrect modular arithmetic.
- Errors in signature verification.

2. Fix Signature Generation

- Generate valid DSA parameters.
- Use cryptographically secure random numbers.
- Verify modular inverse calculations.
- Ensure correct hash computation.
- Validate public and private keys.

3. Optimization

- Use Fast Modular Exponentiation.
- Reduce repeated calculations.
- Optimize modular multiplication.
- Cache reusable values where possible.

4. Impact of Randomness

Strong randomness:

- Prevents private key leakage.
- Produces unique signatures.
- Increases attack resistance.

Weak randomness:

- Repeated signatures.
- Private key exposure.
- Vulnerable to cryptographic attacks.

Conclusion

Correct parameter selection, secure random number generation, and optimized modular arithmetic ensure reliable and secure digital signatures.

• **4. Debugging Key Management System**

Introduction

Key Management Systems (KMS) securely generate, store, distribute, rotate, and destroy cryptographic keys. Poor key management compromises the entire security system.

1. Identify Flaws

- Plaintext key storage.
- No encryption of stored keys.
- Weak access control.
- Missing key rotation.
- Improper backup procedures.

2. Fix Key Lifecycle Management

- Generate strong cryptographic keys.
- Encrypt keys before storage.
- Implement secure distribution.
- Rotate keys periodically.
- Destroy expired keys securely.

3. Optimize Secure Storage

- Store keys using encryption (AES).
- Protect master keys using Hardware Security Modules (HSM).
- Apply Role-Based Access Control (RBAC).
- Maintain audit logs.
- Enable secure backup and recovery.

4. Impact of Poor Key Management

- Data theft.
- Unauthorized decryption.
- Identity impersonation.
- Financial loss.
- Complete system compromise.

Conclusion

Secure storage, encryption, controlled access, and proper key lifecycle management protect cryptographic systems from attacks.

• **5. Optimization of Secure File Transfer Protocol (SFTP-like System)**

Introduction

A Secure File Transfer Protocol (SFTP) encrypts files during transmission. Poor synchronization and packet handling reduce reliability and speed.

1. Debug Encryption/Decryption

- Verify matching encryption and decryption keys.
- Maintain synchronization between sender and receiver.
- Check initialization vectors.

- Validate encryption algorithms.

2. Fix Packet Handling

- Detect missing packets.
- Reassemble packets correctly.
- Verify packet sequence numbers.
- Perform integrity verification using SHA.
- Retransmit corrupted packets.

3. Optimize Throughput

- Use larger packet sizes.
- Compress files before transmission.
- Enable parallel transfer.
- Reduce encryption overhead.
- Buffer file transfers efficiently.

4. Comparison with Insecure File Transfer

Secure File Transfer	Insecure File Transfer
Encrypted data	Plain text data
Integrity verification	No integrity check
User authentication	Weak authentication
Confidential communication	Easily intercepted
Secure against attackers	Vulnerable to attacks

Conclusion

Optimizing encryption, packet handling, and throughput improves transfer speed while maintaining confidentiality, integrity, and authentication.